COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structure and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 16 2026	**DATE SUBMITTED:**	July 16 2026(w/o sup act.) July 21 2026(complete)
NAME:	Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Eng. Jimlord Quejado

Hands-on Activity 2.1 [ACTIVITY NAME]: Arrays, Pointers, and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Table 2-1. Initial Driver Program

INPUT

```
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName ="John Doe",int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

~Student(){
    std::cout << "Distructor Called." << std::endl;
}

Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
```

```
    studentAge = copyStudent.studentAge;
}

void printDetails(){
    std::cout << this ->studentName << " " << this ->studentAge <<
std::endl;
}
};

int main(){
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\CPE010_Regala\test>
IEngine-In-caexbenl.ov2' '--stdout=Microsoft-MI
crt64\bin\gdb.exe' '--interpreter=mi'
Constructor Called.
Copy Constructor Called
Constructor Called.
Distructor Called.
Distructor Called.
Distructor Called.
```

OBSERVATION

The program shows the use of a constructor, copy constructor, destructor, and the default copy assignment operator in a Student class. The output shows that the constructor is called when making student1 and student3, while the copy constructor is called when student2 is initialized using student1. The statement student3 = student2 uses the default copy assignment operator, which is why no copy constructor message is shown. And then, the three destructor messages confirm that each object is automatically destroyed when the program ends, showing proper object lifetime management.

Table 2-2. Modified Driver Program with Student Lists

INPUT

```
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName ="John Doe",int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

~Student(){
    std::cout << "Distructor Called." << std::endl;
}

Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}

void printDetails(){
    std::cout << this ->studentName << " " << this ->studentAge <<
std::endl;
}
};

int main(){
    const size_t j = 5;
```

```
    Student studentList[j] = {};
    std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack",
"Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};


    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Documents\CPE010_Regala\test>
IEngine-In-4wj4drfe.wjd' '--stdout=Microsoft-MI
crt64\bin\gdb.exe' '--interpreter=mi'
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Distructor Called.
Distructor Called.
Distructor Called.
Distructor Called.
Distructor Called.
```

OBSERVATIONS

This code is just the same code as the first one but added some changes to the program from creating individual Student objects to creating an array of five Student objects. Since studentList is declared as an array, the default constructor is automatically called once for each element, which explains why “Constructor Called.” appears five times in the output. Unlike the first program, no copy constructor is invoked because no object is initialized using another existing object, and no copy assignment operation is performed. When the program ends, each object in the array is automatically destroyed in reverse order of creation, resulting in the five “Destructor Called.” messages that confirm proper cleanup of every object.

Table 2-3. Final Driver Program

Loop A	`for(int i = 0; i < j; i++){`
	`Student *ptr = new Student(nameList[i], ageList[i]);`
	`studentList[i] = *ptr;`
	`}`

Observation	My observation is that loop a is the one that generates the constructor called

Loop B	`for(int i = 0; i < j; i++){`
	`studentList[i].printDetails();`
	`}`

Observation	While this loop B generates the destructor called

Output	`PS C:\Users\TIPQC\Documents\CPE010_Regala\test>`
	`IEngine-In-rykvlz4q.o5n' '--stdout=Microsoft-MI`
	`crt64\bin\gdb.exe' '--interpreter=mi'`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Carly 15`
	`Freddy 16`
	`Sam 18`
	`Zack 19`
	`Cody 16`
	`Distructor Called.`
	`Distructor Called.`
	`Distructor Called.`
	`Distructor Called.`
	`Distructor Called.`

Observation	The constructor and destructor were called repeatedly because of the loops

Modifications	`for(int i = 0; i < j; i++){`
	`  Student *ptr = new Student(nameList[i], ageList[i]);`
	`  studentList[i] = *ptr;`
	`  delete ptr;`
	`}`

Observation	The only change I made was adding delete ptr; inside Loop A to properly deallocate the memory that was allocated with new. Since the pointer isn't used again after that point and goes out of scope when the loop ends, setting it to nullptr wasn't necessary.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SCREENSHOT

```
#include <iostream>
#include <string>

class Grocery {
protected:
    std::string name;
    double price;
    int quantity;

public:
    Grocery(std::string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    Grocery(const Grocery &obj) {
        name = obj.name;
        price = obj.price;
        quantity = obj.quantity;
    }

    Grocery& operator=(const Grocery &obj) {
        if (this != &obj) {
```

```
            name = obj.name;
            price = obj.price;
            quantity = obj.quantity;
        }
        return *this;
    }

    virtual ~Grocery() {}

    double calculateSum() const {
        return price * quantity;
    }

    virtual void display() const = 0;
};

class Fruit : public Grocery {
public:
    Fruit(std::string n, double p, int q) : Grocery(n, p, q) {}

    ~Fruit() {}

    void display() const override {
        std::cout << "Fruit: " << name
                  << "\nPrice: PHP " << price
                  << "\nQuantity: " << quantity
                  << "\nSubtotal: PHP " << calculateSum()
                  << "\n\n";
    }
};

class Vegetable : public Grocery {
public:
    Vegetable(std::string n, double p, int q) : Grocery(n, p, q) {}

    ~Vegetable() {}
```

```
    void display() const override {
        std::cout << "Vegetable: " << name
                  << "\nPrice: PHP " << price
                  << "\nQuantity: " << quantity
                  << "\nSubtotal: PHP " << calculateSum()
                  << "\n\n";
    }
};

double TotalSum(Grocery* list[], int size) {
    double total = 0;

    for (int i = 0; i < size; i++) {
        if (list[i] != nullptr)
            total += list[i]->calculateSum();
    }

    return total;
}

int main() {

    Grocery* GroceryList[4];

    GroceryList[0] = new Fruit("Apple", 10, 7);
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);

    std::cout << "Jenna's Grocery List\n\n";

    for (int i = 0; i < 4; i++) {
        GroceryList[i]->display();
    }

    std::cout << "Total Amount: PHP " << TotalSum(GroceryList, 4) <<
std::endl;
```

```
    delete GroceryList[3];
    GroceryList[3] = nullptr;

    std::cout << "\nAfter deleting Lettuce:\n";
    std::cout << "Total Amount: PHP " << TotalSum(GroceryList, 4) <<
std::endl;

    for (int i = 0; i < 4; i++) {
        delete GroceryList[i];
    }

    return 0;
}
```

OUTPUST:

```
Jenna's Grocery List

Fruit: Apple
Price: PHP 10
Quantity: 7
Subtotal: PHP 70

Fruit: Banana
Price: PHP 10
Quantity: 8
Subtotal: PHP 80

Vegetable: Broccoli
Price: PHP 60
Quantity: 12
Subtotal: PHP 720

Vegetable: Lettuce
Price: PHP 50
Quantity: 10
Subtotal: PHP 500

Total Amount: PHP 1370

After deleting Lettuce:
Total Amount: PHP 870
```

Code Analysis:
The program uses object-oriented programming by creating a base Grocery class and two derived classes, Fruit and Vegetable, to organize the grocery items. Each object stores its name, price, and quantity, while the calculateSum() function computes the subtotal for each item and the TotalSum() function calculates the overall cost of the grocery list. Dynamic memory allocation is implemented by creating the objects with new and storing them in an array of pointers, and proper memory management is demonstrated by deleting the Lettuce object first and then deallocating the remaining objects before the program ends to prevent memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.